ROBO-SX

AI + IoT BASED SMART RAILWAY TRACK MONITORING SYSTEM

An intelligent sensing and monitoring concept for early identification of unusual railway track conditions

Project Team	ROBO-SX
Domain	Artificial Intelligence • Internet of Things • Smart Transportation
Core Controller	ESP32
Prototype Sensors	Vibration sensor • DHT11 temperature sensor
Future Processing	Raspberry Pi 5 • Machine Learning (planned)
Submission Version	Technical project report – November 2026

Prepared as a prototype-stage project document. All performance claims are limited to the work completed and observations recorded by the team; railway deployment would require extensive engineering validation and approvals.

CONTENTS

Section	What it covers
1. Executive summary	Project overview, objectives and intended value
2. Problem statement	Railway inspection challenge and project need
3. Proposed solution	System concept and operating workflow
4. Key features and innovation	Core capabilities and differentiators
5. Technology stack	Hardware, software and communication
6. System architecture	Data flow and major modules
7. Implementation	Prototype setup, firmware, dashboard and alert logic
8. Source code and repository	Recommended repository structure and submission checklist
9. Prototype demonstration	Suggested demonstration sequence
10. Dataset and resources	Current data sources and future dataset plan
11. Results and evaluation	Current prototype status and validation metrics
12. Limitations, safety and future scope	Responsible use and roadmap
13. Conclusion and references	Summary and technical resources

1. EXECUTIVE SUMMARY

The ROBO-SX AI + IoT Based Smart Railway Track Monitoring System is a prototype-stage concept designed to demonstrate how low-cost sensing, wireless communication and intelligent data analysis can support railway track condition monitoring. The current prototype uses an ESP32 microcontroller with a vibration sensor and a DHT11 temperature sensor. Sensor readings are intended to be transmitted over Wi-Fi and displayed on a web-based dashboard.

The system is designed to highlight unusual sensor behaviour for further inspection. In its present stage, it is an educational demonstration model—not a certified railway safety device, not a replacement for approved track inspection, and not a system proven to detect cracks or guarantee accident prevention.

The next development phase is planned to add a Raspberry Pi 5 for more capable local processing, structured data storage and machine-learning experiments. A properly labelled dataset, controlled testing and validation against reference measurements will be essential before making any reliability claims.

Project vision: Turn raw sensor readings into useful, understandable condition information that can help maintenance teams decide where further inspection may be needed.

Project objectives

- Build a working educational prototype that reads vibration and temperature sensor signals.
- Transmit sensor data wirelessly from an ESP32 to a monitoring interface.
- Display readings clearly and provide configurable threshold-based warnings.
- Create a foundation for collecting labelled data and testing future machine-learning models.
- Develop the system in stages, with safety, calibration and validation treated as core requirements.

2. PROBLEM STATEMENT

Railway tracks operate under repeated mechanical loading and changing environmental conditions. Track components and their surroundings can experience vibration, temperature variation, wear and other forms of stress. If a developing issue is not identified and assessed in time, it may contribute to service disruption, higher maintenance effort or safety risk.

Conventional inspection and maintenance processes are essential, but they can be resource-intensive and are often performed at scheduled intervals or through specific inspection activities. A low-cost sensing prototype can demonstrate how additional condition data might be gathered between inspections and made easier to review.

Problem statement

There is a need for an affordable, scalable and data-driven supplementary monitoring approach that can collect selected track-environment signals, communicate them to a dashboard and flag unusual readings for human review. The proposed prototype explores this need using ESP32-based IoT sensing and a pathway towards AI-assisted analysis.

Need addressed

- Limited visibility of continuously changing sensor conditions in a simple demonstration setup.
- Need to organise sensor readings in a form that can be reviewed over time.
- Need for early indication of unusual patterns so that qualified personnel can decide whether further inspection is required.
- Need for an affordable learning platform to explore IoT, data analytics and predictive-maintenance concepts.

3. DESCRIPTION OF THE PROPOSED SOLUTION

ROBO-SX proposes a modular monitoring unit built around an ESP32. A vibration sensor provides a signal representing vibration activity at the model or test setup. A DHT11 sensor provides ambient temperature and humidity readings; temperature is the parameter used in the current project concept. The ESP32 reads the sensor outputs and sends available readings through Wi-Fi to a web dashboard.

The dashboard is intended to present live or periodically refreshed values, a system status and warning messages when a reading crosses a configured threshold. Threshold warnings are treated as indicators for review—not as proof of a defect. The next phase will focus on collecting properly labelled measurements, filtering noise, extracting useful features and evaluating machine-learning methods for distinguishing normal from unusual patterns.

Operating workflow

Stage	Operation	Output
1. Sensing	Vibration and temperature sensors capture signals from the demonstration setup.	Raw sensor readings
2. Acquisition	ESP32 reads sensor values and applies basic checks.	Timestamped readings (where implemented)
3. Communication	ESP32 sends available data over Wi-Fi.	Data received by dashboard endpoint
4. Visualisation	Dashboard presents sensor values and system status.	Human-readable monitoring view
5. Alert logic	Configured rules flag readings outside selected limits.	Advisory warning for review
6. Future intelligence	A validated dataset may be analysed on Raspberry Pi 5 using ML.	Experimental anomaly classification

4. KEY FEATURES AND INNOVATION

Feature	Description	Current status
Vibration sensing	Captures vibration activity from the model/test arrangement.	Sensor response observed; dashboard integration and calibration require completion/verification.
Temperature monitoring	Reads ambient temperature using DHT11.	Included in prototype setup; accuracy should be checked against a reference.
ESP32 connectivity	Provides sensor acquisition and Wi-Fi capability.	Core platform selected and used in prototype.
Web dashboard	Presents sensor information in an accessible interface.	Dashboard prototype developed; verify each live data channel before claiming full integration.
Threshold alerts	Highlights values that cross configured limits.	Proposed/subject to end-to-end testing.
AI-assisted analysis	Future analysis of labelled sensor patterns.	Planned; no trained model or measured AI accuracy claimed at this stage.
Modular expansion	Supports future sensors and edge-processing upgrades.	Design direction.

What makes the concept innovative

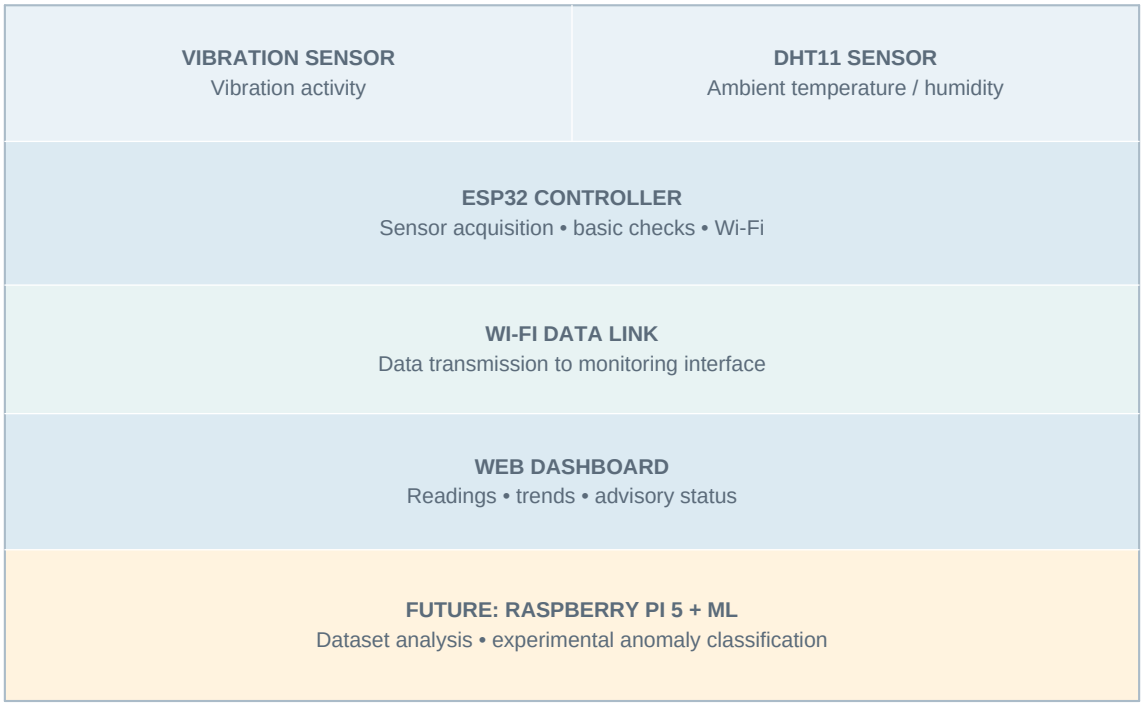
- **Layered development:** begins with accessible ESP32 sensing and creates a clear path towards edge computing and AI.
- **Human-in-the-loop alerts:** warnings are designed to support inspection decisions rather than make unsafe autonomous decisions.
- **Data-first approach:** the project emphasises collecting and labelling measurements before claiming machine-learning performance.
- **Modular architecture:** additional sensors, location tagging and data storage can be added without redesigning the entire concept.
- **Low-cost learning platform:** enables students to explore railway monitoring concepts through embedded systems, IoT and data analytics.

5. TECHNOLOGY STACK

Layer	Technology	Role in the project
Controller	ESP32	Reads sensor inputs and provides Wi-Fi connectivity.
Vibration sensing	Vibration sensor module	Detects vibration activity; sensor output depends on module type and mounting.
Environmental sensing	DHT11	Measures ambient temperature and humidity.
Firmware	Embedded C/C++	Controls sensor sampling, basic processing and communication.
IDE	Arduino IDE	Used to develop and upload ESP32 firmware.
Connectivity	Wi-Fi / HTTP or other selected protocol	Transfers data from the ESP32 to the dashboard.
Dashboard	HTML, CSS, JavaScript (as applicable)	Displays readings and system status in a browser.
Future data analysis	Python (planned)	Data cleaning, visualisation and model experimentation.
Future edge device	Raspberry Pi 5 (planned)	Potential local data processing and ML inference.
Future ML	Scikit-learn / TensorFlow Lite (to be evaluated)	Candidate tools; selection depends on dataset size and model needs.

6. SYSTEM ARCHITECTURE

The proposed architecture separates sensing, communication, visualisation and future intelligence. This makes the system easier to test one module at a time.



Data flow: Sensors → ESP32 → Wi-Fi → Dashboard. The Raspberry Pi 5 and machine-learning stage is a planned extension and is not represented as an already deployed component.

7. BRIEF EXPLANATION OF IMPLEMENTATION

7.1 Hardware assembly

Connect the vibration sensor output to a suitable ESP32 input and connect the DHT11 according to its module pinout. Use a stable power supply and common ground where required. Confirm the voltage and signal-level requirements of each module before wiring. Mount sensors consistently, because changes in mounting position can significantly affect vibration readings.

7.2 Firmware development

The ESP32 firmware initialises the sensors, reads values at a defined sampling interval, checks for invalid readings and prepares the data for transmission. For vibration signals, a single digital trigger may provide only limited information; a more meaningful analysis may require an analogue or higher-rate sensor, repeated sampling and signal processing. The exact method depends on the sensor hardware used.

7.3 Data transmission and dashboard

The ESP32 connects to a Wi-Fi network and sends readings to the dashboard using the communication method implemented by the team. The dashboard should show the sensor name, value, unit, last-update time and connection state. If data is missing, the interface should show 'No data' or 'Connection unavailable' rather than displaying an old value as if it were live.

7.4 Alert logic

For the prototype, threshold-based rules can be used to mark values that fall outside a chosen test range. Thresholds must be selected through controlled experiments and should not be presented as official railway safety limits. Alerts should be labelled 'Advisory – inspection required' and must not trigger operational railway decisions.

7.5 Future AI development

A future AI workflow should start by collecting repeated readings under documented test conditions. Each record should include a timestamp, sensor configuration, mounting position, operating condition and a reliable label. After cleaning and splitting the dataset into training, validation and test sets, candidate algorithms can be evaluated. Model performance should be reported using precision, recall, F1-score, confusion matrix and false-alarm rate—not accuracy alone.

8. SOURCE CODE / REPOSITORY

The source code should be maintained in a version-controlled repository such as GitHub. The following structure is recommended; adapt the filenames to match the actual code written by the team.

```
ROBO-SX-Rail-Monitoring/
■■■ firmware/
■ ■■■ esp32_sensor_monitor.ino
■■■ dashboard/
■ ■■■ index.html
■ ■■■ style.css
■ ■■■ script.js
■■■ data/
■ ■■■ sample_readings.csv
■■■ documentation/
■ ■■■ project_report.pdf
■ ■■■ wiring_diagram.png
■■■ media/
■ ■■■ prototype_photo.jpg
■ ■■■ demo_video_link.txt
■■■ README.md
```

Repository URL: [Insert actual GitHub repository link]

README.md should explain the project, hardware requirements, wiring, software setup, Wi-Fi configuration method, how to run the dashboard, known limitations and safety disclaimer. Never upload Wi-Fi passwords, API keys or other private credentials.

Repository submission checklist

- Upload the actual ESP32 firmware and dashboard source files.
- Add a wiring diagram and list the exact sensor module models.
- Include sample data only if it was genuinely collected by the team.
- Document the steps needed to reproduce the demonstration.
- Add a licence and acknowledge external libraries or code used.

9. WORKING PROTOTYPE / DEMONSTRATION

The physical model is intended to demonstrate sensor acquisition and wireless monitoring. The demonstration should be limited to the model/test setup and should not be performed on an operational railway track.

Recommended demonstration sequence

Step	Demonstration action	What to show
1	Power on the prototype.	ESP32 startup and sensor connection status.
2	Show the temperature sensor reading.	DHT11 value on the dashboard and its update time.
3	Apply a safe, repeatable vibration to the demonstration model.	Vibration sensor response at the sensor output or serial monitor.
4	Show the dashboard data path.	Confirm whether the vibration reading reaches and updates on the dashboard.

Step	Demonstration action	What to show
5	Demonstrate an advisory threshold warning, if implemented and tested.	Warning appears for a deliberately simulated test value.
6	Explain the next phase.	Raspberry Pi 5, labelled dataset and ML validation roadmap.

Important prototype-status note: Based on the team's development notes, the vibration sensor has shown a physical response, while its display on the dashboard still requires end-to-end integration and verification. Do not claim that live vibration data is fully visible on the dashboard until this is confirmed in the actual demonstration.

Evidence to attach

- Photographs of the complete prototype and close-ups of sensor connections.
- A short unedited video showing the sensor response and dashboard update.
- A screenshot of the dashboard with timestamped, real readings.
- A short serial-monitor capture or test log to support sensor verification.

10. DATASET AND EXTERNAL RESOURCES

Current data source

The prototype is based on readings collected directly from the connected sensors. No independently verified railway defect dataset or trained AI model is claimed in this report. If a sample CSV is attached, it should be clearly identified as actual sensor data, simulated data or example data.

Recommended future dataset fields

Field	Description
timestamp	Date and time of each reading.
vibration_raw / vibration_feature	Raw sensor output or a documented derived feature.
temperature_c	Temperature reading in degrees Celsius.
humidity_percent	DHT11 humidity reading, if used.
sensor_id and mounting	Sensor model, location on test rig and mounting details.
test_condition	Documented test condition, such as baseline or controlled vibration level.
label	Verified label assigned from the controlled test protocol; avoid guessing real fault labels.

External technical resources to consult

- Espressif ESP32 documentation: https://docs.espressif.com/
- Arduino documentation: https://docs.arduino.cc/
- Raspberry Pi documentation: https://www.raspberrypi.com/documentation/
- Python documentation: https://docs.python.org/3/
- Scikit-learn documentation: https://scikit-learn.org/stable/

List all libraries, code examples, datasets, images and third-party resources actually used in the final submission, with their source links and licences. The list above is a suggested technical reference list, not a claim that every resource has already been used.

11. RESULTS AND EVALUATION

Current prototype-stage observations

- A physical railway monitoring model has been developed to demonstrate the project concept.
- ESP32 has been selected as the main controller for sensor acquisition and Wi-Fi connectivity.
- A vibration sensor has shown a physical response during testing; dashboard display of that signal requires further integration and verification.
- A DHT11 temperature sensor is included in the prototype setup; its readings should be checked against a reference thermometer.
- A web dashboard prototype has been developed; each live data channel and alert function should be verified before being reported as fully operational.

Evaluation plan

Metric	How to evaluate	Evidence to record
Temperature error	Compare DHT11 readings with a reference thermometer at several stable conditions.	Reference value, sensor value, absolute error.
Vibration response	Repeat the same controlled test with fixed sensor mounting and multiple trials.	Raw output, sampling rate, repeatability.
Data delivery rate	Send a known number of readings and count successful dashboard updates.	Sent, received and missed records.
End-to-end latency	Measure time between sensor acquisition and dashboard display.	Timestamped samples and measured delay.
Alert behaviour	Apply controlled test values around the configured threshold.	Correct alerts, missed alerts and false alerts.
Future model quality	Evaluate on a separate held-out test set with verified labels.	Precision, recall, F1-score, confusion matrix.

Results reporting template

After testing, replace the placeholders below with measured values. Do not invent performance figures.

Parameter	Measured result
Temperature reference comparison	[To be measured]
Vibration detection repeatability	[To be measured]
Successful dashboard updates	[To be measured]
Average data-update delay	[To be measured]
Threshold alert test result	[To be measured]
AI model metrics	Not available yet – ML model planned

12. LIMITATIONS, SAFETY AND FUTURE SCOPE

Current limitations

- A low-cost demonstration sensor does not directly measure rail integrity or prove the presence of a crack.
- Vibration readings depend on sensor type, mounting, sampling rate, train/model conditions and environmental noise.

- DHT11 is an entry-level environmental sensor and is not a rail-temperature safety instrument.
- A dashboard warning based on a threshold is not equivalent to a validated fault-detection system.
- The project has not been validated on live railway infrastructure and must not be installed on or tested near operational tracks.

Safety and responsible-use statement

This project is an academic prototype for demonstration and research. It is not certified, safety-rated or approved for railway operations. It must not be used to make train movement, signalling, maintenance clearance or emergency decisions. Any real-world pilot would require railway-authority approval, qualified engineering supervision, certified equipment, a formal risk assessment, cybersecurity review and extensive testing under applicable standards.

Future development roadmap

Phase	Planned work	Deliverable
Phase 1 – Stabilise prototype	Verify sensor wiring, calibration, sampling and reliable dashboard updates.	Repeatable ESP32 + sensor demonstration.
Phase 2 – Data logging	Add timestamps, CSV/database storage and test metadata.	Documented, labelled experimental dataset.
Phase 3 – Edge computing	Integrate Raspberry Pi 5 for local processing and data management.	ESP32-to-Pi data pipeline.
Phase 4 – AI experimentation	Train and compare anomaly-detection/classification models using validated data.	Model evaluation report with limitations.
Phase 5 – Controlled validation	Test on a safe laboratory rig and compare against reference instruments.	Repeatability and performance evidence.
Phase 6 – Expert review	Seek feedback and permissions from railway-domain experts and relevant authorities.	Feasibility assessment for any approved pilot.

13. CONCLUSION

ROBO-SX presents an IoT-enabled railway monitoring concept that combines ESP32-based sensing, wireless data communication and a web dashboard, with a planned path towards edge processing and AI. The project addresses the value of gathering sensor information in a structured and accessible way so unusual patterns can be reviewed by people responsible for inspection and maintenance.

The immediate priority is to complete and verify the end-to-end sensor-to-dashboard pipeline, document repeatable tests and build a reliable dataset. Raspberry Pi 5 and machine learning are future extensions that should be introduced only after sufficient high-quality data and validation methods are available. With careful development, the project can evolve from an educational prototype into a stronger research platform for condition-monitoring concepts.

APPENDIX: FINAL SUBMISSION CHECKLIST

- Project report updated with actual team member names, mentor details and date.
- Actual GitHub repository URL inserted and repository made accessible to evaluators.
- Prototype photos and a short demonstration video attached.
- Screenshots show real sensor values, timestamps and connection status.
- Known incomplete features are clearly identified.
- Dataset origin, labels, licences and external resources are documented.

- Measured evaluation results replace all placeholders.
- Safety disclaimer is retained in the report and presentation.

END OF REPORT • ROBO-SX